

▸ AUTONOMOUS_RESEARCH_LOOP // RLAIIF ON A VISION-LANGUAGE MODEL

48 HOURS. ONE **AI** RESEARCHER.

I ran a research lab for two days. The only scientist was an AI agent — teaching a vision model to stop making things up. One AI trains, another AI grades, and no human ever labels an answer.

RESEARCHERS

1 agent

HARDWARE

1 GPU

GRADING BUDGET

\$0.00

FAITHFUL TO IMAGE

50 → 72%

VISION AI LIES — FLUENTLY.

It states what isn't in the image as confidently as what is — the #1 trust problem for any real product.

The goal: a small open-source model (3B params — one gaming GPU), made **measurably more faithful** — with no human grading answers.

When AI grades AI, nobody notices when it **breaks — unless you build the instruments.**

SAME PHOTO • OUR MODEL'S OWN ANSWER • BEFORE VS AFTER TRAINING



▣ held-out photo • "What voltage is that outlet rated at?"

STEP 100

"Rated at **120 volts** — standard for US homes."

STEP 500

"I **can't determine** the voltage from the image — you'd need to check the plug."

Source : [RLAIF-V dataset](#) • [openbmb](#) • [HuggingFace](#) — the model's own probe answers, training step 100 → 500.

THREE LEVELS OF RLAIIF — WE DID TWO, ON PURPOSE

LET AN AI GRADE THE AI.

LEVEL 1 • DID ✓

Offline RLAIIF

Train on an existing AI-labeled preference dataset — no live loop.

DPO • PAIRWISE A/B • OFFLINE

LEVEL 2 • DID ✓

Closed-loop RLAIIF

Our model writes answers → a separate AI judge grades → we retrain. \$0, free judge. The level that earns the name.

DPO • PAIRWISE A/B • OFFLINE

LEVEL 3 • NOT YET ✕

Online RL

Reward every rollout live. Needs fast infra and a low-noise reward — our judge flips 20% of verdicts, and level 2 already showed what optimizing that noise does.

PPO • ONLINE • SCALAR REWARD → ADVANTAGES • KL

How we keep score: held-out preference accuracy — does the model rank the faithful answer above the hallucinated one? 50% = chance (± 3 at $n=256$). Measured, never optimized — that's what makes it evidence, not memorization.

THE CAST, AND THE CONTRACT

WHO DID WHAT.

HUMAN – LOUIS

- Sets the direction & the goal
- Sets the quality bars
- Interrupts, and demands proofs

AGENT – CLAUDE

- Researches & writes the code
- Trains, evaluates, monitors
- Reports – and self-corrects

I never wrote a line of code or watched a training run. I asked questions and demanded proofs.

DAY 0 → THE WHOLE RUN

RESEARCH → BUILD → LOOP.

- 1** Research. Claude ran parallel deep-research – the 80/20, the evals, a hackathon roadmap – before a line of code.
- 2** Build the MVP. Not a model – the lab: infra + evaluation, wired from day 0 so nothing runs unmeasured.
- 3** `/loop` autoresearch. Pick the next experiment → run it → conclude in report.md → repeat.

One axis at a time – model, then data, then feedback – so every gain was attributable.

INSTRUMENTS BEFORE EXPERIMENTS

THE FIRST DELIVERABLE WASN'T A MODEL.

● ● ● smic - the lab, in one command each

```
$ make deploy    # train on the remote GPU, in tmux
$ make monitor   # live Trackio dashboard - curves + outputs
$ make verify    # prove the training is bug-free
$ make sweep     # run experiments in parallel
$ make report    # regenerate every figure in this deck
```

**No experiment
without a **measurement**.**

Every result writes **twice** – to the live dashboard and to committed files – so **report.md** regenerates every figure and **lessons.md** banks every gotcha. Any number here rebuilds from scratch.

YOU STEER BY **PROMPT.**

- 0 "Keep going." a cron nudge every few minutes so the loop never stalls unattended.
- 1 "Prove the training is bug-free." the overfit test – loss must start at 0.693 and fall to 0. Caught a "frozen" part still training.
- 2 "Use the whole GPU – run experiments in parallel." a memory trick (same gradient, $\frac{1}{3}$ the footprint) ran 3 trials at once → utilization 40% → 98%.
- 3 "Reports must interpret the science, not report uptime." every check-in became a diagnosis vs previous runs.
- 4 "Compute and API are fixed budgets." pre-registered gates + ship-best-checkpoint; a free-tier judge with hard caps → \$0 of grading.

THE AUTONOMY LOOP

FOR THE LAST 20 HOURS, NO ONE WAS WATCHING.



+ SELF-RELAUNCHING CHAINS

one 3B run fills the GPU, so experiments ran **serially**
– each launching the next:
train → eval → decide → scale → overnight → eval

+ EVERY RESULT, COMMITTED

science to **report.md**, gotchas to **lessons.md** – the
loop's own memory

THE AGENT'S LAB NOTEBOOK

WRITTEN WHILE THE LAB SLEPT.

[18:40] qwen3b-final-16k → acc 0.701 – best result yet

[20:05] evening eval suite → judge noise measured (20%), retrain, win-rates

[21:35] qwen3b-overnight-24k LAUNCHED – full epoch, 3× the data

[23:30] training... gates read every ~40 min through the night

[~02:00] flat phase... held, not killed (the epoch-aware gate)

[overnight] step 2500 → acc 0.709 ★ NEW PROJECT RECORD – while everyone slept

[08:00] morning self-evaluation + final figures

[10:00] committed. done.

IT CAUGHT THREE BUGS IN **ITS OWN** WORK.

- 1** The moving exam. Different experiments were being graded on different test questions. → **pinned one shared test set for every run.**
- 2** The unreliable judge. Ask the judge the same question with the two answers swapped, and its verdict flips 20% of the time. → **measured the noise, then a 3-vote majority protocol.**
- 3** The contaminated exam. The overnight model scored an impossible 0.887... because the test set had leaked into its training data. → **caught in minutes (too good to be true), re-graded clean → 0.723.**

The most important agent skill isn't intelligence. It's **self-suspicion.**

THE SCOREBOARD – "DOES THE MODEL RANK THE FAITHFUL ANSWER ABOVE THE HALLUCINATED ONE?" · 50% = CHANCE

FROM COIN-FLIP TO 72%.

50%

a coin flip
learned nothing



58%

end of day 1
the 500M model



72.3%

final model, verified-clean
(vs 65.6% runner-up)

WHAT MOVED THE NEEDLE, IN ORDER – MEASURED ON SHARED TEST SETS

- 1 **+8** fixed the **learning rate** – a sweep found it ~200× too low (500M)
- 2 **+8** scaled to 3B + **recipe overhaul** + 4× data – bundled under deadline
- 3 **+1** 2× more data – accuracy flat (within ±3 noise), ranking **confidence** +43%
- 4 **+7** 3× data + a **full epoch** – on the clean, never-trained set → 72.3%

Honest read: per-component gains weren't separated – under deadline, scale + recipe + data shipped together; earlier own-slice numbers ran hot (the "moving exam" bug). 72.3 vs 65.6 is the clean comparison – and every scaled run went **flat**, then **jumped**, so a naive stop-rule would have killed the winner.

CLOSED-LOOP RLAIIF

IT IMPROVED — THEN ATE ITS OWN TAIL.

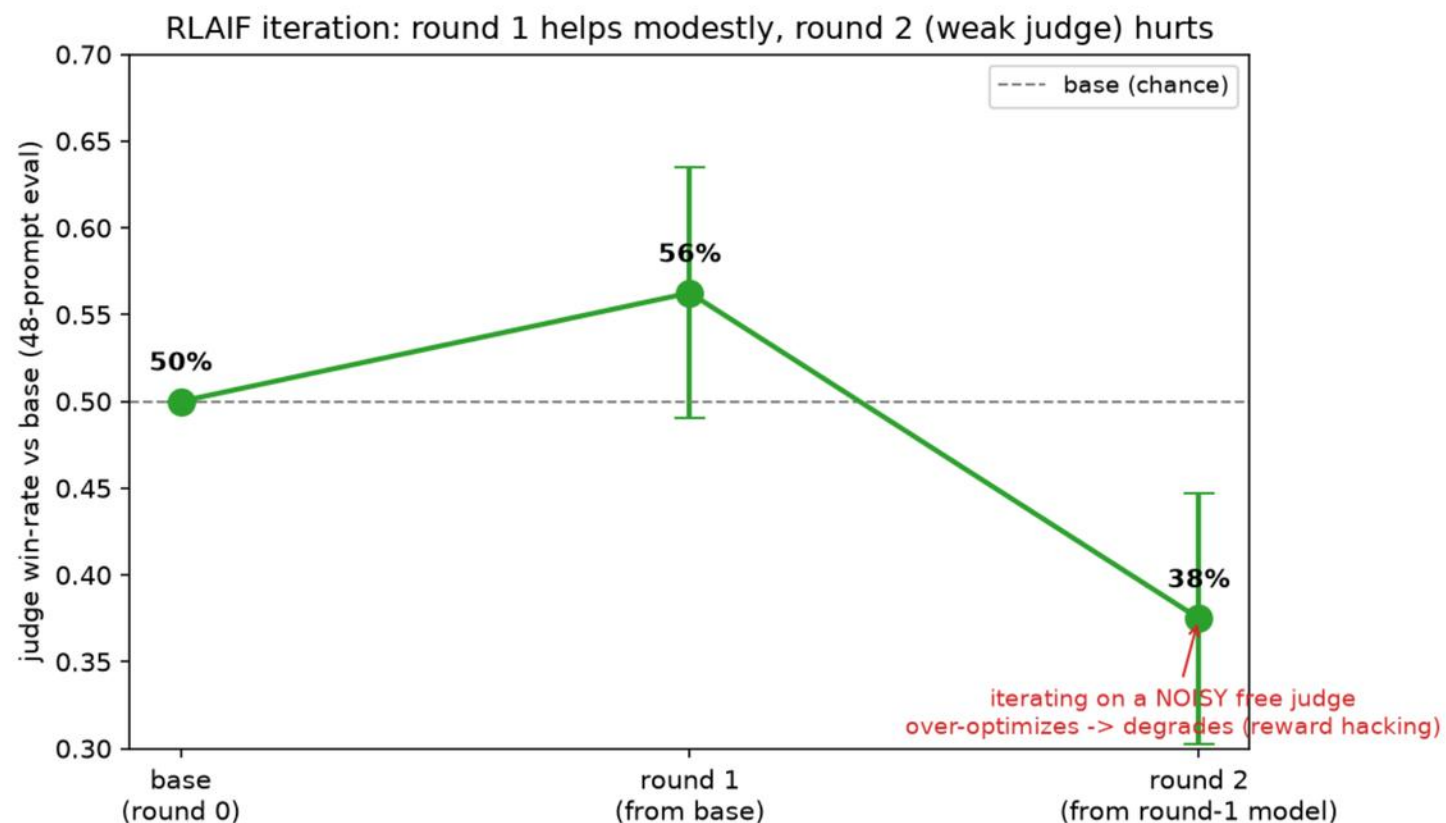
The judge is a **separate** free vision model – not the one being trained. Push too hard against its **noisy** verdicts and the model games the judge instead of getting better.

56% Round 1 – studies the judge's critiques of its own answers, gets modestly better.

38% Round 2 – iterate again and it's worse than the start. It over-optimized a noisy teacher.

58% De-noise the feedback (3-vote judge) → the gain returns. **The judge's quality is the ceiling.**

● ● ● ▶ make rlaif · rlaif_iteration.png



48 HOURS. ONE GPU. \$0.

WALL-CLOCK

48h

one weekend

HARDWARE

1GPU

gaming-class, 24 GB

AI GRADING

\$0

free-tier judge, budget-guarded

STAFF

1AI

the only researcher

LAB NOTEBOOK

95

commits, timestamped through the night

WHAT GENERALIZES TO YOUR WORK

FROM THE HUMAN

- ▶ Set the direction
- ▶ Set the quality bars
- ▶ Demand proofs, not status

FROM THE AGENT

- ▶ Build the instruments first
- ▶ Pre-register the rules
- ▶ Suspect your own good news

► END_OF_RUN

WHAT WOULD YOU DELEGATE TO AN AGENT FOR 48 HOURS?

The method worked end-to-end on tiny hardware – but the real product is **the way the lab ran itself**: direction and proofs from the human; instruments, pre-registered rules, and self-suspicion from the agent.

STORY.md · PRESENTATION.md · every number traceable to a committed experiment